Modele nieliniowe

Statystyka w SI i analizie danych — laboratorium 7

Wojciech Maćkowiak, 412430

2026-04-21

Contents

1 Regresja wielomianowa **2**

2 Wykres dopasowania z pasmami błędu standardowego **3**

3 Regresja logistyczna wielomianowa **4**

4 Funkcje schodkowe **6**

5 Funkcje sklejane — `bs()` **7**
5.1 Wpływ położenia węzłów . . . 8
5.2 Wpływ liczby stopni swobody (`df`) dla `bs` . . . 10
5.3 Stopień wielomianów w `bs` (`degree`) . . . 11

6 Naturalne funkcje sklejane — `ns()` **12**
6.1 Wpływ `df` dla `ns` . . . 13

7 Wygładzające funkcje sklejane — `smooth.spline` **13**

8 Regresja lokalna — `loess` **15**

9 Uogólnione modele addytywne (GAM) **17**
9.1 GAM dopasowany metodą najmniejszych kwadratów (`lm` + `ns`) . . . 17
9.2 GAM — algorytm wsteczny, pakiet `gam` . . . 19
9.3 Porównanie modeli — `anova` . . . 20
9.4 GAM z lokalną regresją `lo()` . . . 21

10 GAM w GLM — regresja logistyczna **22**

Pakiety: `ISLR` (zbiór `Wage`), `splines` (`bs`, `ns`), `gam` (`gam`, `s`, `lo`, `plot.Gam`). Instalacja: `install.packages(c("ISLR", "gam"))` (pakiet `splines` jest w bazie R).

```
library(ISLR)
library(splines)
library(gam)
data(Wage)
age_lims <- range(Wage$age)
age_grid <- seq(age_lims[1], age_lims[2])
```

1 Regresja wielomianowa

Regresja wielomianowa stopnia 4: `wage` względem `age`.

```r
fit_poly <- lm(wage ~ poly(age, 4), data = Wage)
summary(fit_poly)
```

```
##
## Call:
## lm(formula = wage ~ poly(age, 4), data = Wage)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -98.707 -24.626  -4.993  15.217 203.693
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)    111.7036     0.7287 153.283  < 2e-16 ***
## poly(age, 4)1  447.0679    39.9148  11.201  < 2e-16 ***
## poly(age, 4)2 -478.3158    39.9148 -11.983  < 2e-16 ***
## poly(age, 4)3  125.5217    39.9148   3.145  0.00168 **
## poly(age, 4)4  -77.9112    39.9148  -1.952  0.05104 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 39.91 on 2995 degrees of freedom
## Multiple R-squared:  0.08626,    Adjusted R-squared:  0.08504
## F-statistic: 70.69 on 4 and 2995 DF,  p-value: < 2.2e-16
```

To samo ze standardową bazą wielomianów X, X^2, X^3, X^4 (`raw = TRUE`).

```r
fit_poly_raw <- lm(wage ~ poly(age, 4, raw = TRUE), data = Wage)
summary(fit_poly_raw)
```

```
##
## Call:
## lm(formula = wage ~ poly(age, 4, raw = TRUE), data = Wage)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -98.707 -24.626  -4.993  15.217 203.693
##
## Coefficients:
##                             Estimate Std. Error t value Pr(>|t|)
## (Intercept)               -1.842e+02  6.004e+01  -3.067 0.002180 **
## poly(age, 4, raw = TRUE)1  2.125e+01  5.887e+00   3.609 0.000312 ***
## poly(age, 4, raw = TRUE)2 -5.639e-01  2.061e-01  -2.736 0.006261 **
## poly(age, 4, raw = TRUE)3  6.811e-03  3.066e-03   2.221 0.026398 *
## poly(age, 4, raw = TRUE)4 -3.204e-05  1.641e-05  -1.952 0.051039 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 39.91 on 2995 degrees of freedom
## Multiple R-squared:  0.08626,    Adjusted R-squared:  0.08504
## F-statistic: 70.69 on 4 and 2995 DF,  p-value: < 2.2e-16
```

Ekwiwalent zapisany przez `I(age^k)`:

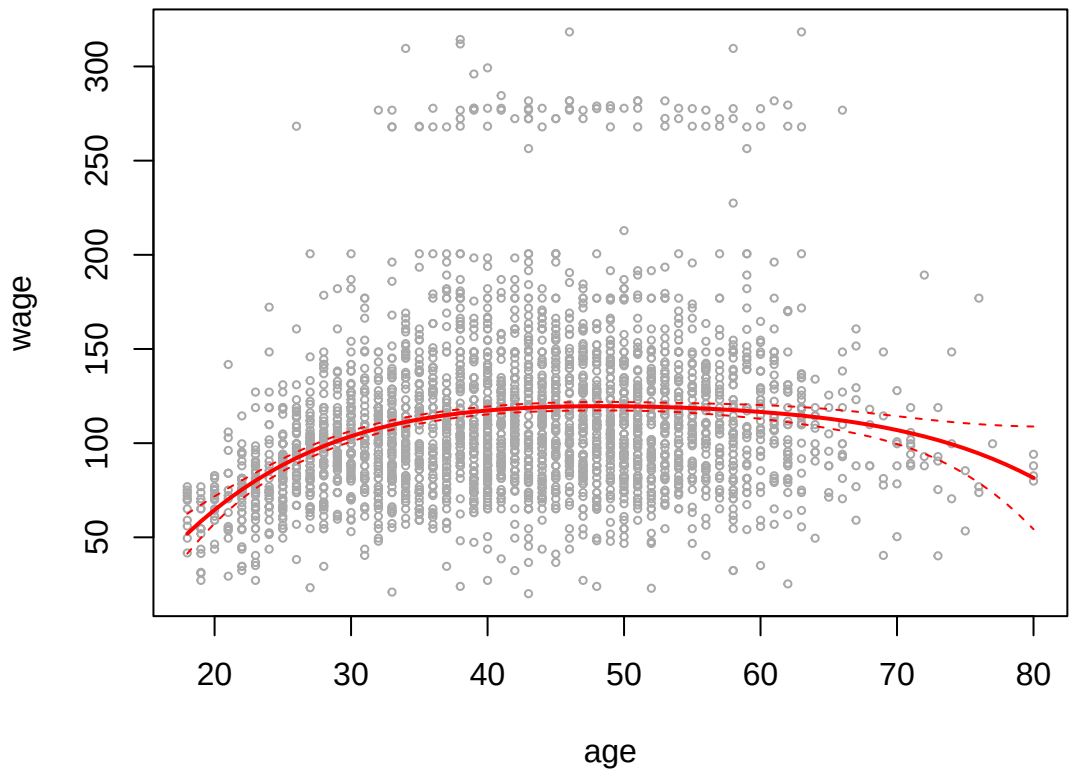
```r
fit_poly_raw2 <- lm(wage ~ age + I(age^2) + I(age^3) + I(age^4), data = Wage)
summary(fit_poly_raw2)
```

```
##
## Call:
## lm(formula = wage ~ age + I(age^2) + I(age^3) + I(age^4), data = Wage)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -98.707 -24.626  -4.993  15.217 203.693
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.842e+02  6.004e+01  -3.067 0.002180 **
## age          2.125e+01  5.887e+00   3.609 0.000312 ***
## I(age^2)    -5.639e-01  2.061e-01  -2.736 0.006261 **
## I(age^3)     6.811e-03  3.066e-03   2.221 0.026398 *
## I(age^4)    -3.204e-05  1.641e-05  -1.952 0.051039 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 39.91 on 2995 degrees of freedom
## Multiple R-squared:  0.08626,    Adjusted R-squared:  0.08504
## F-statistic: 70.69 on 4 and 2995 DF,  p-value: < 2.2e-16
```

Wersja ortogonalna (`poly` bez `raw`) ułatwia interpretację testów kolejnych stopni; surowe współczynniki odpowiadają bezpośrednio potęgom `age`.

2 Wykres dopasowania z pasmami błędu standardowego

```r
pred_poly <- predict(fit_poly, list(age = age_grid), se.fit = TRUE)
se_bands <- cbind(
  pred_poly$fit + 2 * pred_poly$se.fit,
  pred_poly$fit - 2 * pred_poly$se.fit
)
plot(Wage$age, Wage$wage, col = "darkgrey", cex = 0.5, xlim = age_lims,
     xlab = "age", ylab = "wage")
lines(age_grid, pred_poly$fit, col = "red", lwd = 2)
matlines(age_grid, se_bands, col = "red", lty = "dashed")
```



3 Regresja logistyczna wielomianowa

Klasyfikacja: duże zarobki (`wage > 250`, skala jak w ISLR) vs pozostali; predyktor `age` z potęgami do 4.

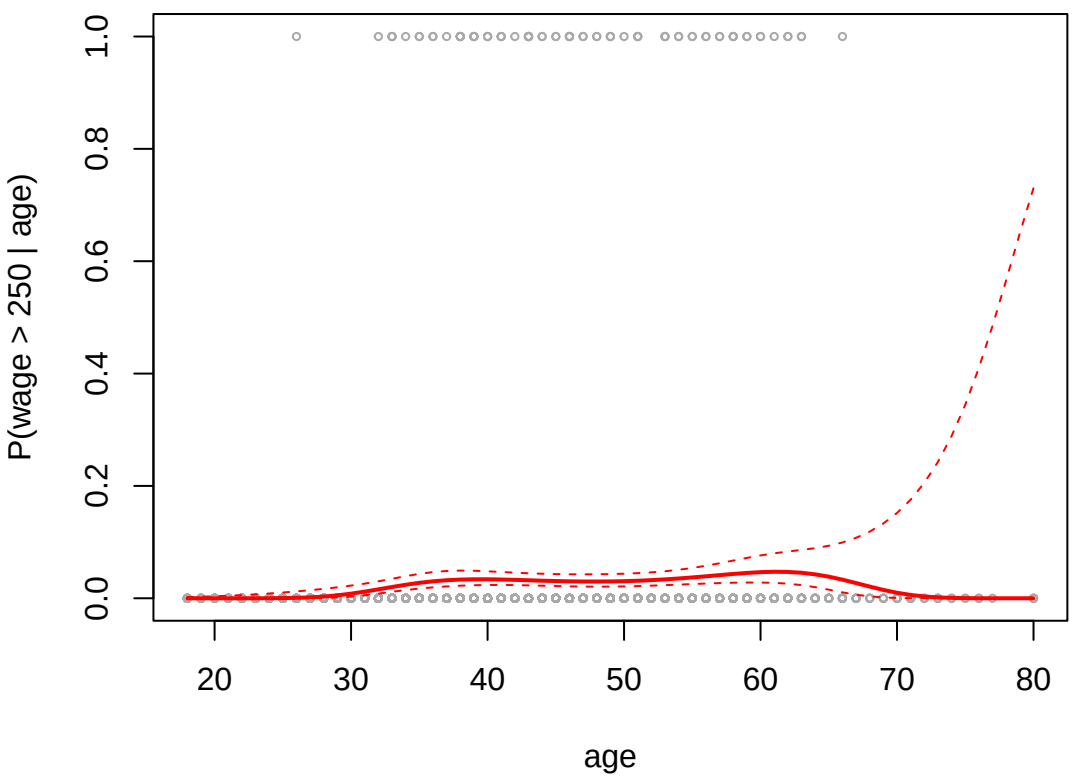
```
fit_log_poly <- glm(I(wage > 250) ~ poly(age, 4), data = Wage, family = binomial)
summary(fit_log_poly)
```

```
##
## Call:
## glm(formula = I(wage > 250) ~ poly(age, 4), family = binomial,
##     data = Wage)
##
## Coefficients:
##                Estimate Std. Error z value Pr(>|z|)
## (Intercept)     -4.3012     0.3451 -12.465  < 2e-16 ***
## poly(age, 4)1   71.9642    26.1176   2.755  0.00586 **
## poly(age, 4)2  -85.7729    35.9043  -2.389  0.01690 *
## poly(age, 4)3   34.1626    19.6890   1.735  0.08272 .
## poly(age, 4)4  -47.4008    24.0909  -1.968  0.04912 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
```

```
##
##     Null deviance: 730.53  on 2999  degrees of freedom
## Residual deviance: 701.22  on 2995  degrees of freedom
## AIC: 711.22
##
## Number of Fisher Scoring iterations: 9
```

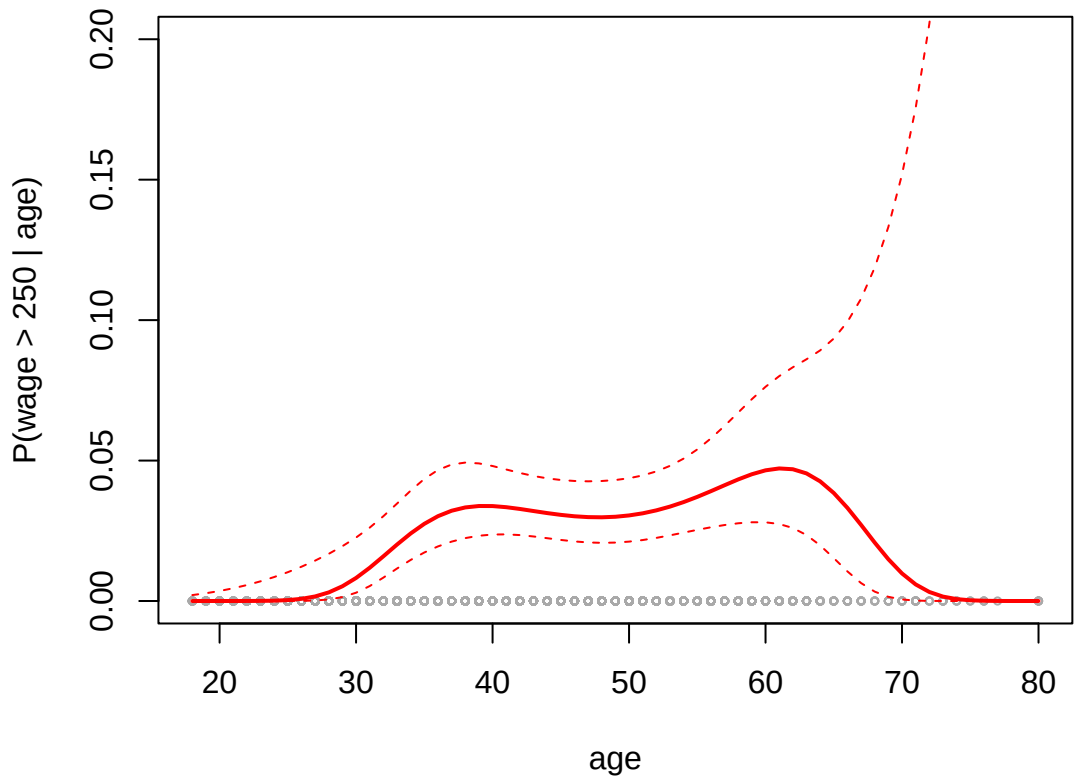
`predict.glm` na skali logitu — pasma SE przekształcamy `plogis`.

```r
pred_log_poly <- predict(fit_log_poly, list(age = age_grid), se.fit = TRUE)
pred_probs <- plogis(pred_log_poly$fit)
se_bands_logit <- cbind(
  pred_log_poly$fit + 2 * pred_log_poly$se.fit,
  pred_log_poly$fit - 2 * pred_log_poly$se.fit
)
se_bands_log <- plogis(se_bands_logit)
plot(Wage$age, I(Wage$wage > 250), xlim = age_lims, ylim = c(0, 1),
     col = "darkgrey", cex = 0.5, ylab = "P(wage > 250 | age)", xlab = "age")
lines(age_grid, pred_probs, col = "red", lwd = 2)
matlines(age_grid, se_bands_log, lty = "dashed", col = "red")
```



„Powiększenie” — np. `ylim = c(0, 0.2)` — ułatwia zobaczenie małych prawdopodobieństw w środkowym wieku.

```r
plot(Wage$age, I(Wage$wage > 250), xlim = age_lims, ylim = c(0, 0.2),
     col = "darkgrey", cex = 0.5, ylab = "P(wage > 250 | age)", xlab = "age")
lines(age_grid, pred_probs, col = "red", lwd = 2)
matlines(age_grid, se_bands_log, lty = "dashed", col = "red")
```



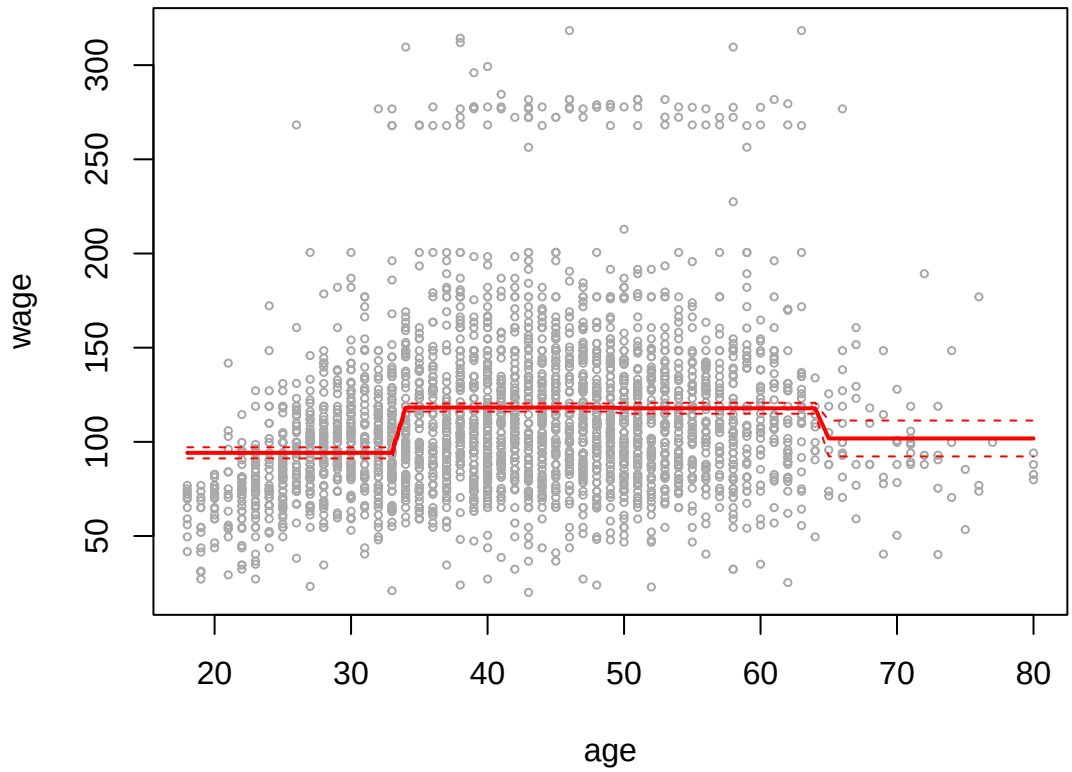
4 Funkcje schodkowe

```r
table(cut(Wage$age, breaks = 4))
```

```
##
## (17.9,33.5]   (33.5,49]   (49,64.5] (64.5,80.1]
##         750        1399         779          72
```

```r
fit_step <- lm(wage ~ cut(age, 4), data = Wage)
pred_step <- predict(fit_step, list(age = age_grid), se.fit = TRUE)
se_bands_step <- cbind(
  pred_step$fit + 2 * pred_step$se.fit,
  pred_step$fit - 2 * pred_step$se.fit
)
plot(Wage$age, Wage$wage, col = "darkgrey", cex = 0.5, xlim = age_lims,
     xlab = "age", ylab = "wage")
lines(age_grid, pred_step$fit, col = "red", lwd = 2)
```

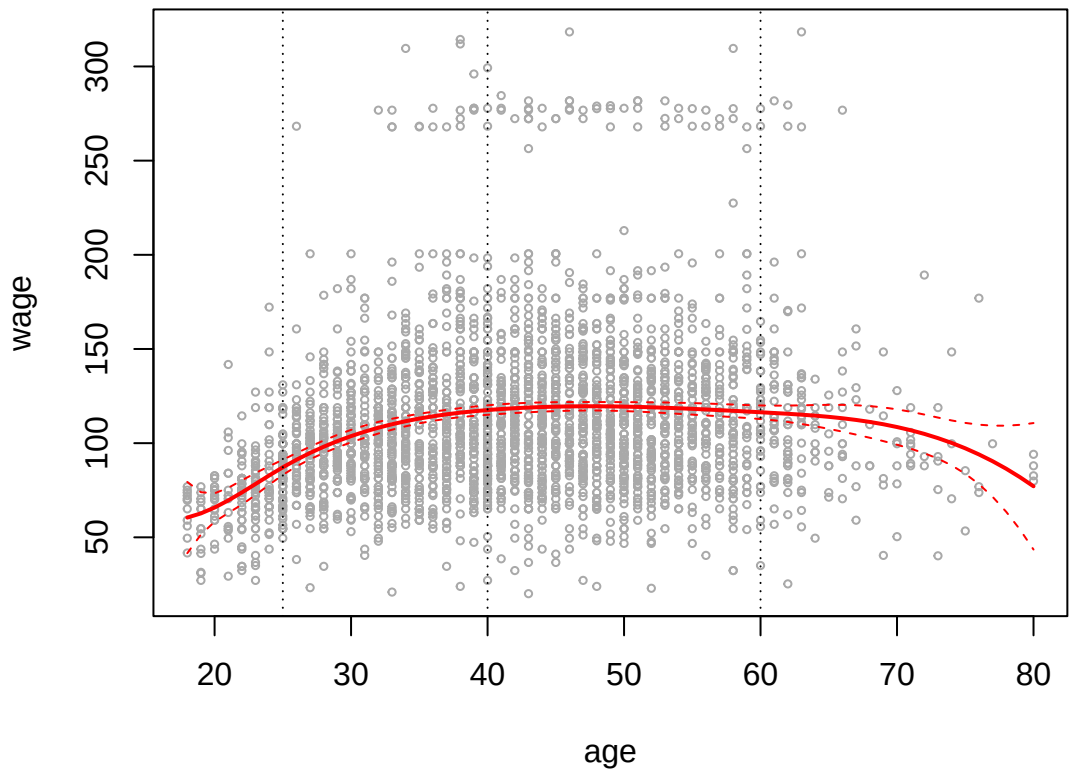
```
matlines(age_grid, se_bands_step, col = "red", lty = "dashed")
```



5 Funkcje sklejane — `bs()`

Domyślny stopień splajnu to 3. Model z ustalonymi węzłami:

```
fit_bs_knots <- lm(wage ~ bs(age, knots = c(25, 40, 60)), data = Wage)
pred_bs_knots <- predict(fit_bs_knots, list(age = age_grid), se.fit = TRUE)
plot(Wage$age, Wage$wage, cex = 0.5, col = "darkgrey", xlab = "age", ylab = "wage")
lines(age_grid, pred_bs_knots$fit, col = "red", lwd = 2)
lines(age_grid, pred_bs_knots$fit + 2 * pred_bs_knots$se.fit, col = "red", lty = "dashed")
lines(age_grid, pred_bs_knots$fit - 2 * pred_bs_knots$se.fit, col = "red", lty = "dashed")
abline(v = c(25, 40, 60), lty = "dotted")
```



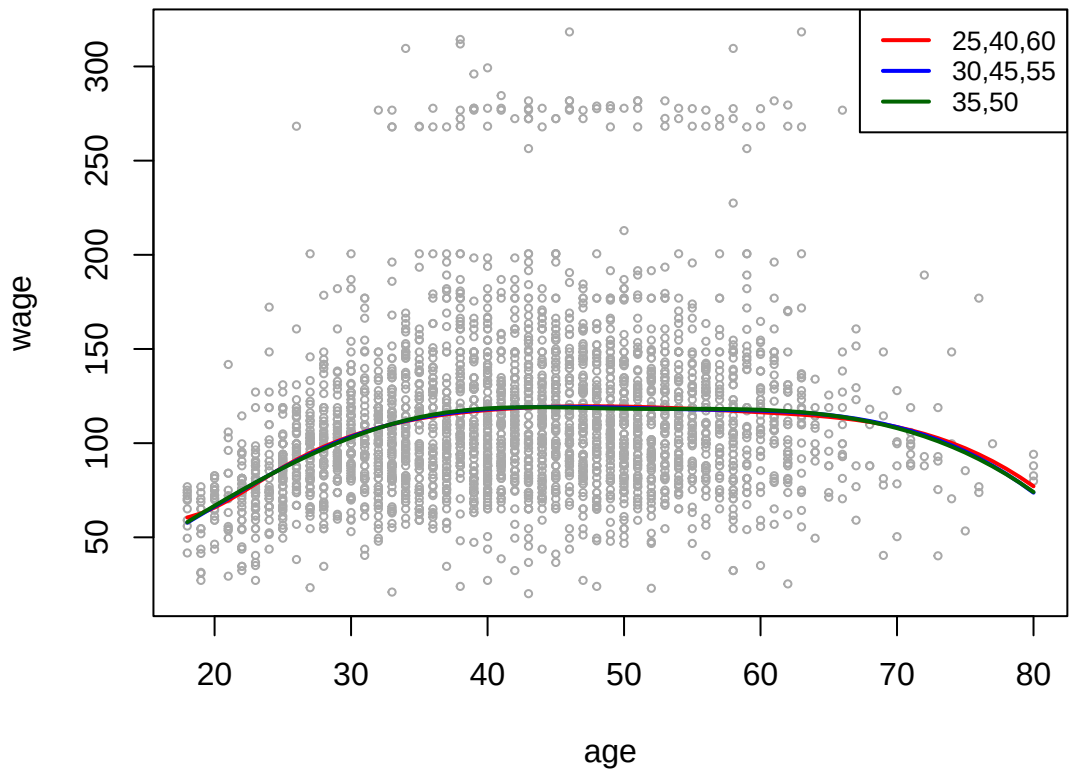
5.1 Wpływ położenia węzłów

Porównanie dwóch zestawów węzłów wewnętrznych + `AIC` (niższe — lepsze dopaszenie z karą za złożoność):

```
k_sets <- list(
  c(25, 40, 60),
  c(30, 45, 55),
  c(35, 50)
)
fits_k <- lapply(k_sets, function(k) lm(wage ~ bs(age, knots = k), data = Wage))
aic_k <- sapply(fits_k, AIC)
names(aic_k) <- vapply(k_sets, function(k) paste(k, collapse = ","), "")
aic_k
```

```
## 25,40,60 30,45,55    35,50
## 30644.59 30643.06 30639.97
```

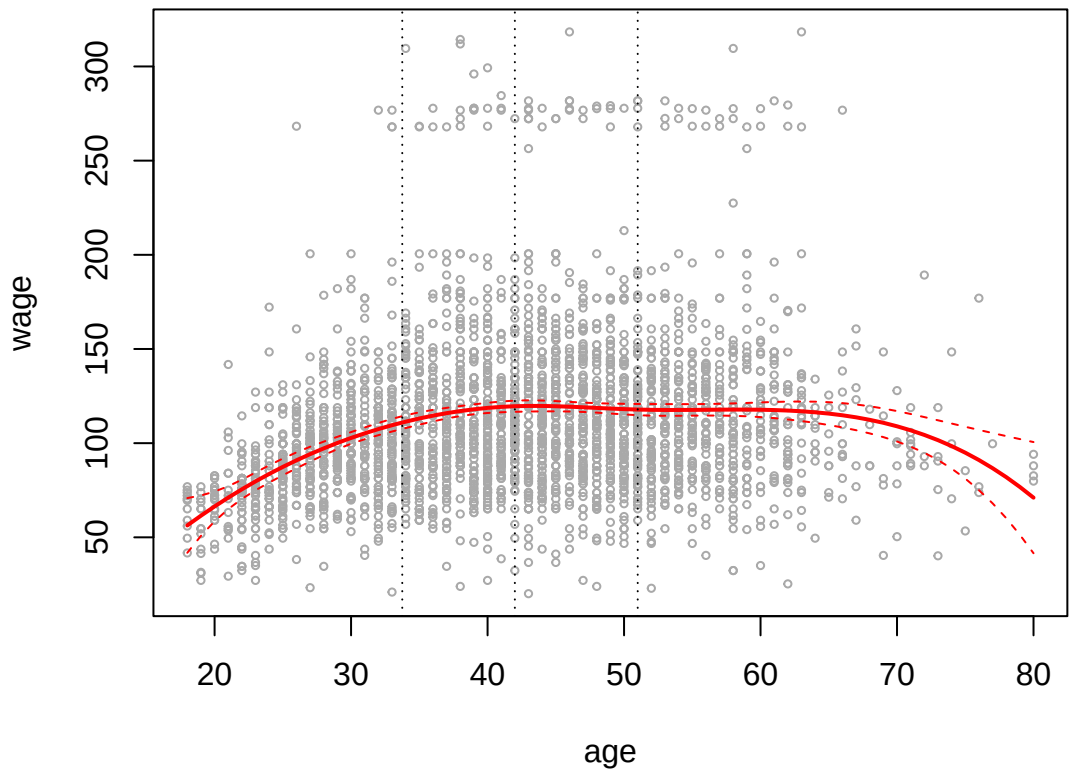
```
plot(Wage$age, Wage$wage, cex = 0.5, col = "darkgrey", xlab = "age", ylab = "wage")
cols <- c("red", "blue", "darkgreen")
for (i in seq_along(fits_k)) {
  p <- predict(fits_k[[i]], list(age = age_grid))
  lines(age_grid, p, col = cols[i], lwd = 2)
}
legend("topright", legend = names(aic_k), col = cols, lty = 1, lwd = 2, cex = 0.8)
```



Inne węzły zmieniają lokalizację „załamań” krzywej; `AIC` pozwala porównać kompromis dopasowanie–złożoność między specyfikacjami o tej samej liczbie baz.

Model z automatycznym rozmieszczeniem węzłów przy ustalonej liczbie stopni swobody:

```
fit_bs_df <- lm(wage ~ bs(age, df = 6), data = Wage)
pred_bs_df <- predict(fit_bs_df, list(age = age_grid), se.fit = TRUE)
plot(Wage$age, Wage$wage, cex = 0.5, col = "darkgrey", xlab = "age", ylab = "wage")
lines(age_grid, pred_bs_df$fit, col = "red", lwd = 2)
lines(age_grid, pred_bs_df$fit + 2 * pred_bs_df$se.fit, col = "red", lty = "dashed")
lines(age_grid, pred_bs_df$fit - 2 * pred_bs_df$se.fit, col = "red", lty = "dashed")
bs_knots <- attr(bs(Wage$age, df = 6), "knots")
abline(v = bs_knots, lty = "dotted")
```

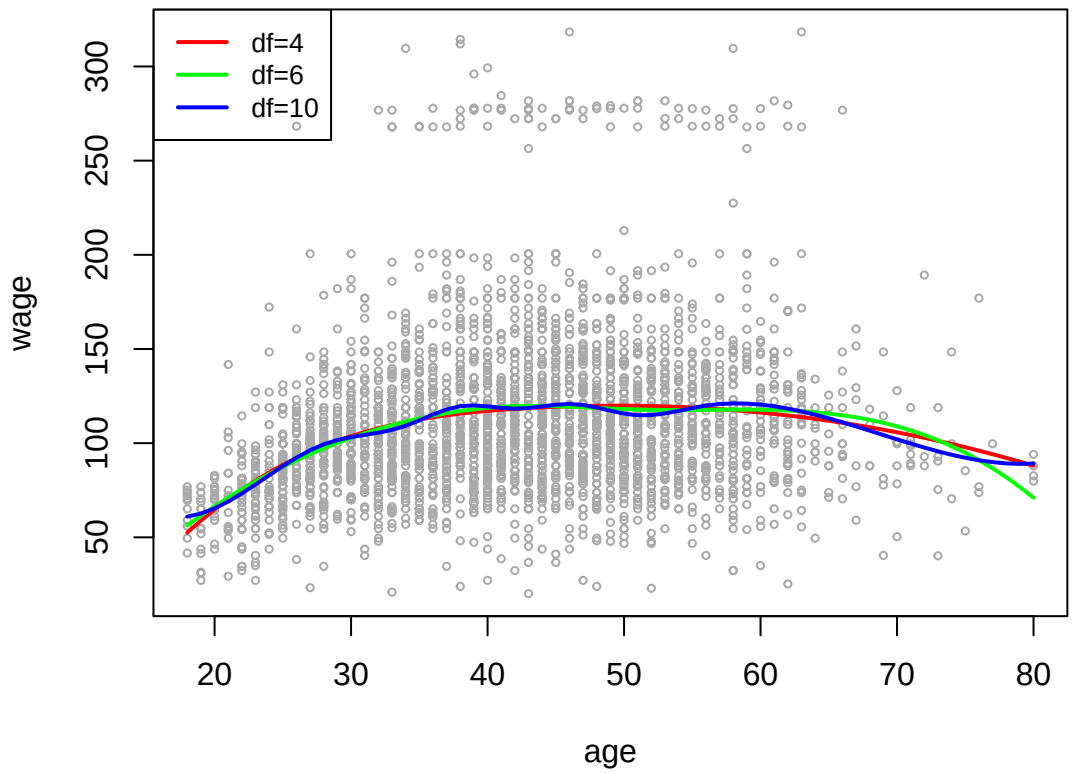


5.2 Wpływ liczby stopni swobody (df) dla bs

```
dfs <- c(4, 6, 10)
fits_bs_df <- lapply(dfs, function(d) lm(wage ~ bs(age, df = d), data = Wage))
aic_bs <- sapply(fits_bs_df, AIC)
names(aic_bs) <- paste0("df=", dfs)
aic_bs
```

```
##     df=4     df=6    df=10
## 30642.05 30641.74 30642.03
```

```
plot(Wage$age, Wage$wage, cex = 0.5, col = "darkgrey", xlab = "age", ylab = "wage")
cl <- rainbow(length(dfs))
for (i in seq_along(dfs)) {
  pr <- predict(fits_bs_df[[i]], list(age = age_grid))
  lines(age_grid, pr, col = cl[i], lwd = 2)
}
legend("topleft", legend = names(aic_bs), col = cl, lty = 1, lwd = 2, cex = 0.85)
```



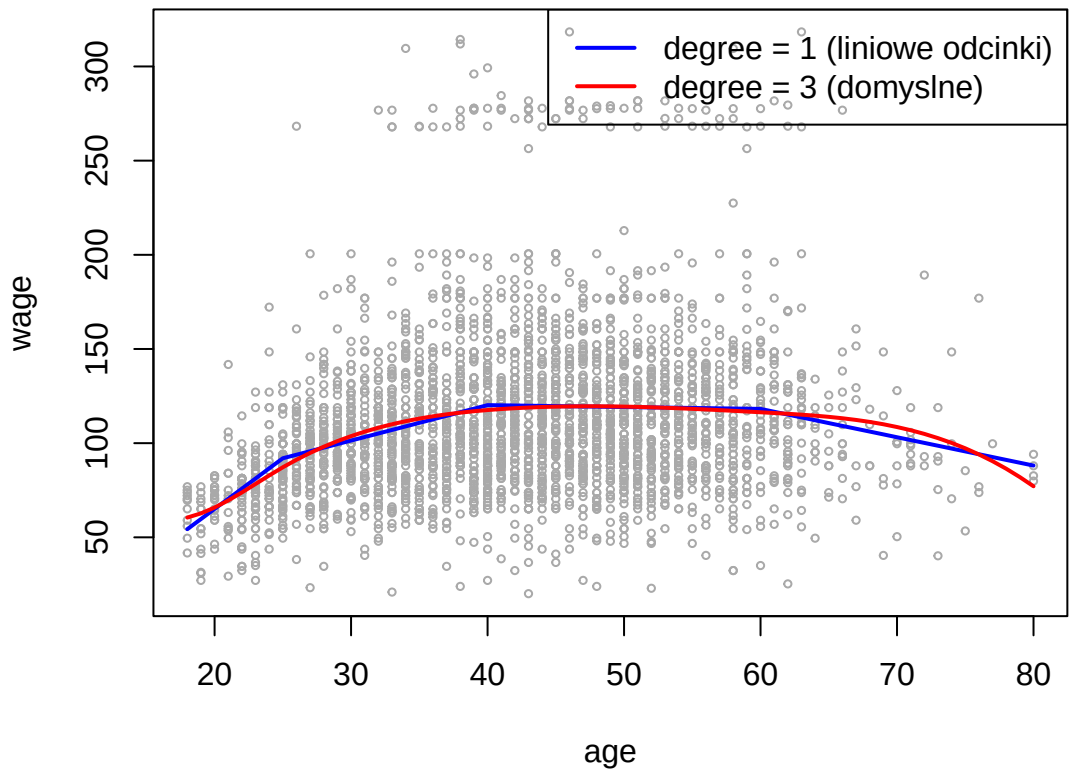
Większe `df` — bardziej „giętka” krzywa i zwykle niższe RSS na próbie, przy ryzyku przeuczenia.

5.3 Stopień wielomianów w `bs` (`degree`)

```
fit_d1 <- lm(wage ~ bs(age, knots = c(25, 40, 60), degree = 1), data = Wage)
fit_d3 <- lm(wage ~ bs(age, knots = c(25, 40, 60), degree = 3), data = Wage)
AIC(fit_d1, fit_d3)
```

```
##        df      AIC
## fit_d1  6 30639.85
## fit_d3  8 30644.59
```

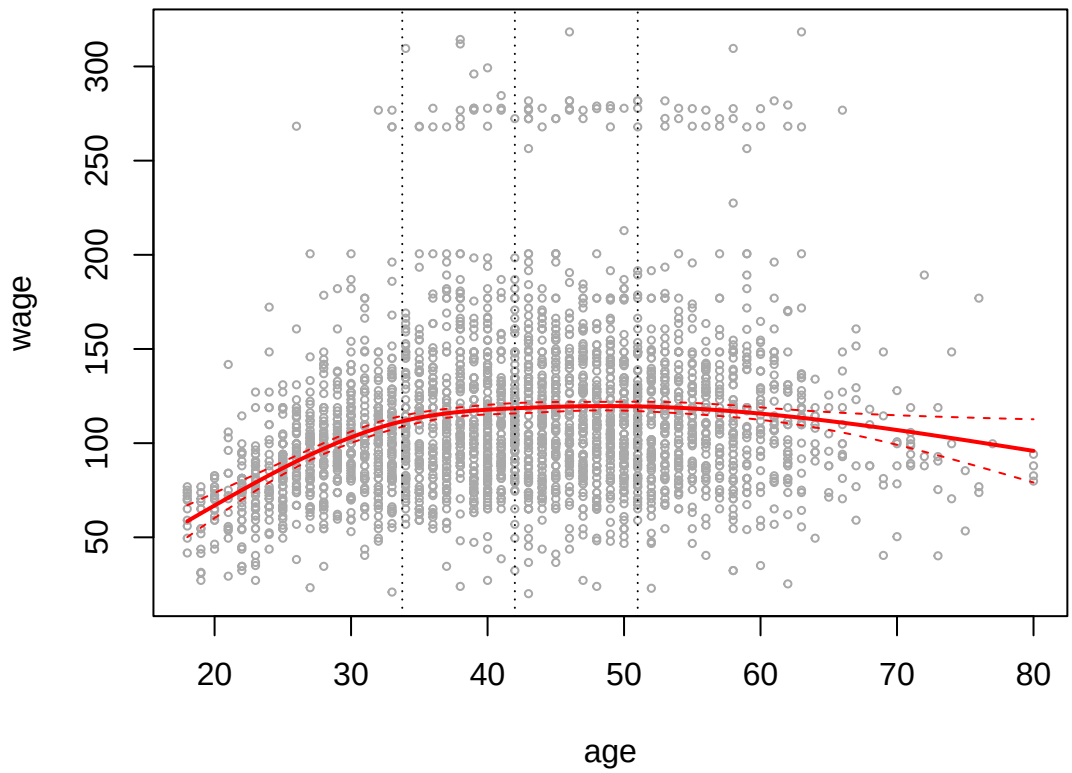
```
plot(Wage$age, Wage$wage, cex = 0.5, col = "darkgrey", xlab = "age", ylab = "wage")
lines(age_grid, predict(fit_d1, list(age = age_grid)), col = "blue", lwd = 2)
lines(age_grid, predict(fit_d3, list(age = age_grid)), col = "red", lwd = 2)
legend("topright", legend = c("degree = 1 (liniowe odcinki)", "degree = 3 (domyślne)"),
       col = c("blue", "red"), lty = 1, lwd = 2)
```



`degree = 1` daje ciągłą, „łamaną” krzywą liniową między węzłami; wyższy stopień zwiększa gładkość między węzłami.

6 Naturalne funkcje sklejane — `ns()`

```
fit_ns <- lm(wage ~ ns(age, df = 4), data = Wage)
pred_ns <- predict(fit_ns, list(age = age_grid), se.fit = TRUE)
plot(Wage$age, Wage$wage, cex = 0.5, col = "darkgrey", xlab = "age", ylab = "wage")
lines(age_grid, pred_ns$fit, col = "red", lwd = 2)
lines(age_grid, pred_ns$fit + 2 * pred_ns$se.fit, col = "red", lty = "dashed")
lines(age_grid, pred_ns$fit - 2 * pred_ns$se.fit, col = "red", lty = "dashed")
abline(v = attr(ns(Wage$age, df = 4), "knots"), lty = "dotted")
```



6.1 Wpływ `df` dla `ns`

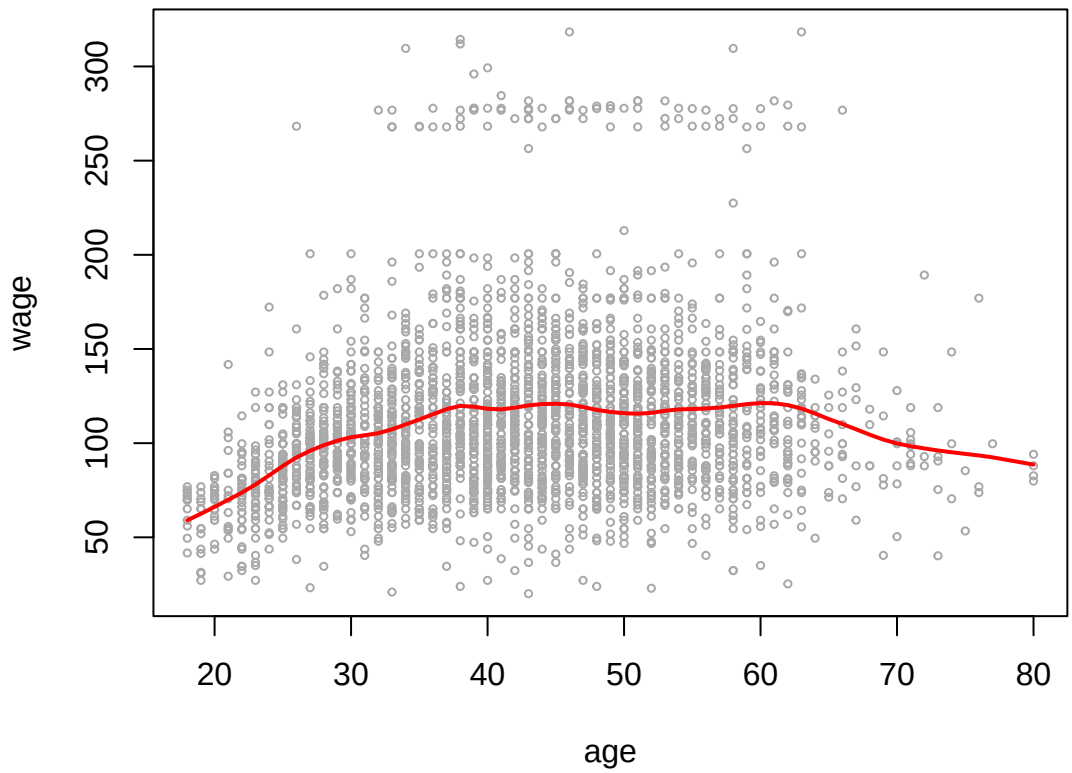
```
dfs_ns <- c(3, 4, 6)
fits_ns <- lapply(dfs_ns, function(d) lm(wage ~ ns(age, df = d), data = Wage))
aic_ns <- sapply(fits_ns, AIC)
names(aic_ns) <- paste0("df=", dfs_ns)
aic_ns
```

```
##     df=3     df=4     df=6
## 30641.39 30642.05 30640.17
```

Naturalne splajny dodatkowo narzucają liniowość poza zakresem danych — często stabilniejsze na brzegach niż zwykłe `bs`.

7 Wygładzające funkcje sklejane — `smooth.spline`

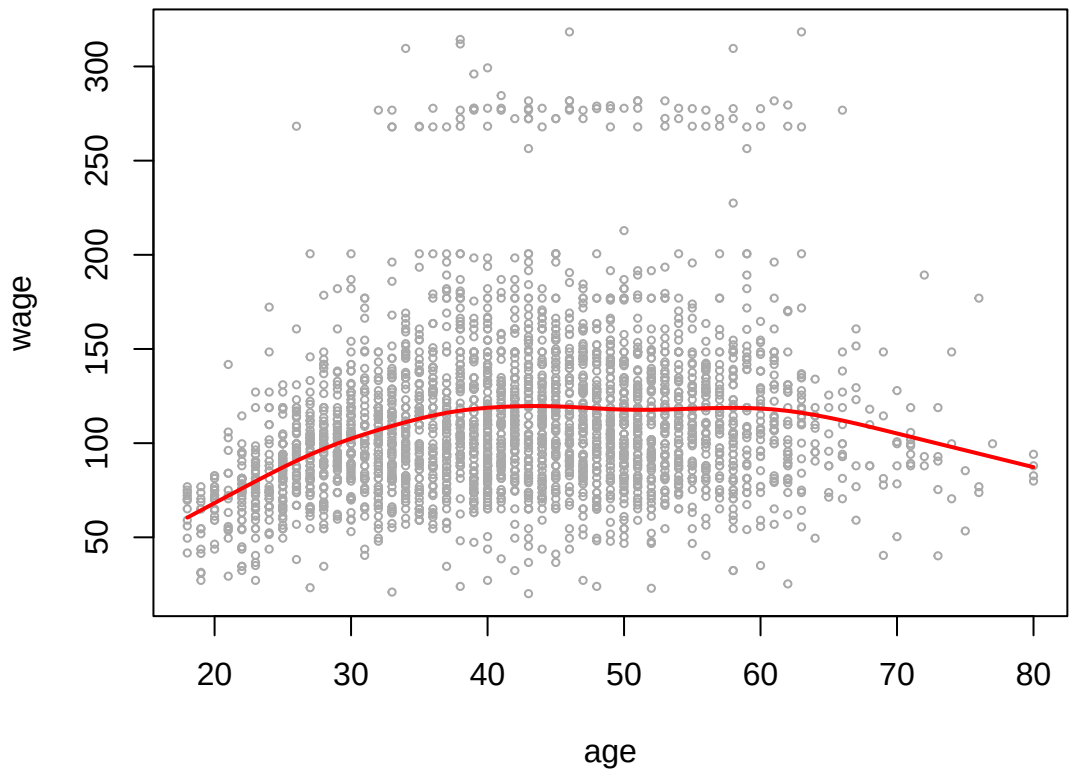
```
fit_smooth_df <- smooth.spline(Wage$age, Wage$wage, df = 16)
plot(Wage$age, Wage$wage, cex = 0.5, col = "darkgrey", xlab = "age", ylab = "wage")
lines(fit_smooth_df, col = "red", lwd = 2)
```




```r
fit_smooth_cv <- smooth.spline(Wage$age, Wage$wage, cv = TRUE)
fit_smooth_cv$df
```

```
## [1] 6.794596
```

```r
plot(Wage$age, Wage$wage, cex = 0.5, col = "darkgrey", xlab = "age", ylab = "wage")
lines(fit_smooth_cv, col = "red", lwd = 2)
```

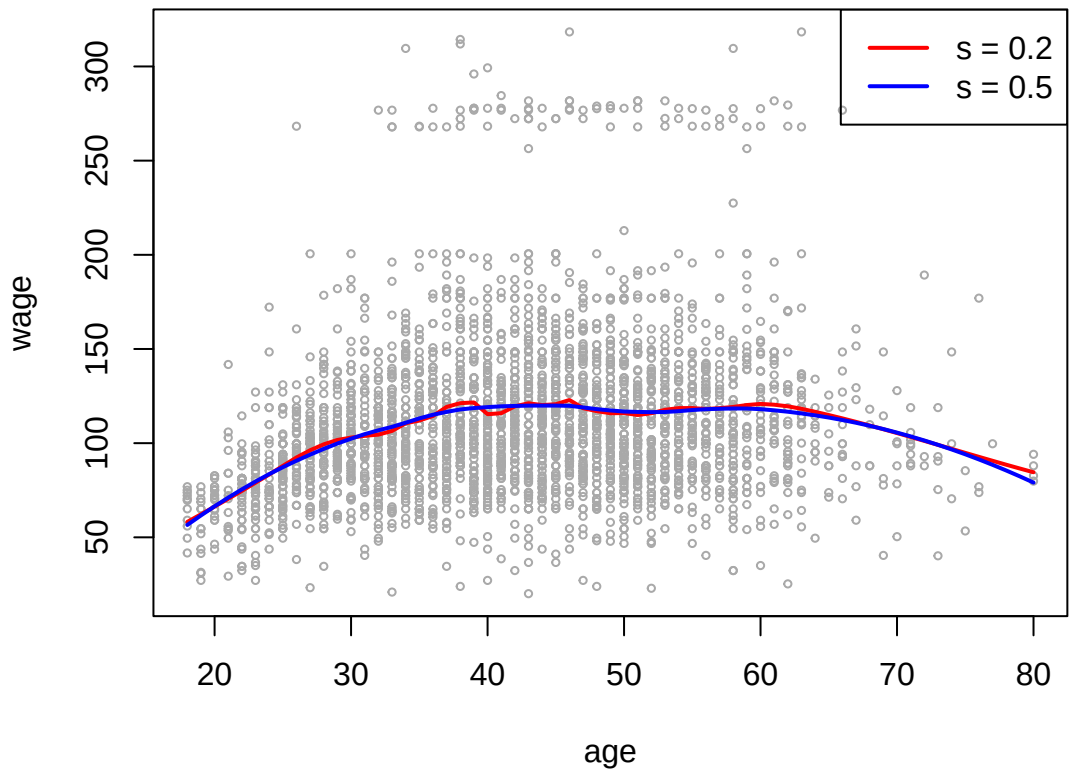


Walidacja krzyżowa wybiera efektywną liczbę stopni swobody (tu około wartości wypisanej powyżej).

8 Regresja lokalna — `loess`

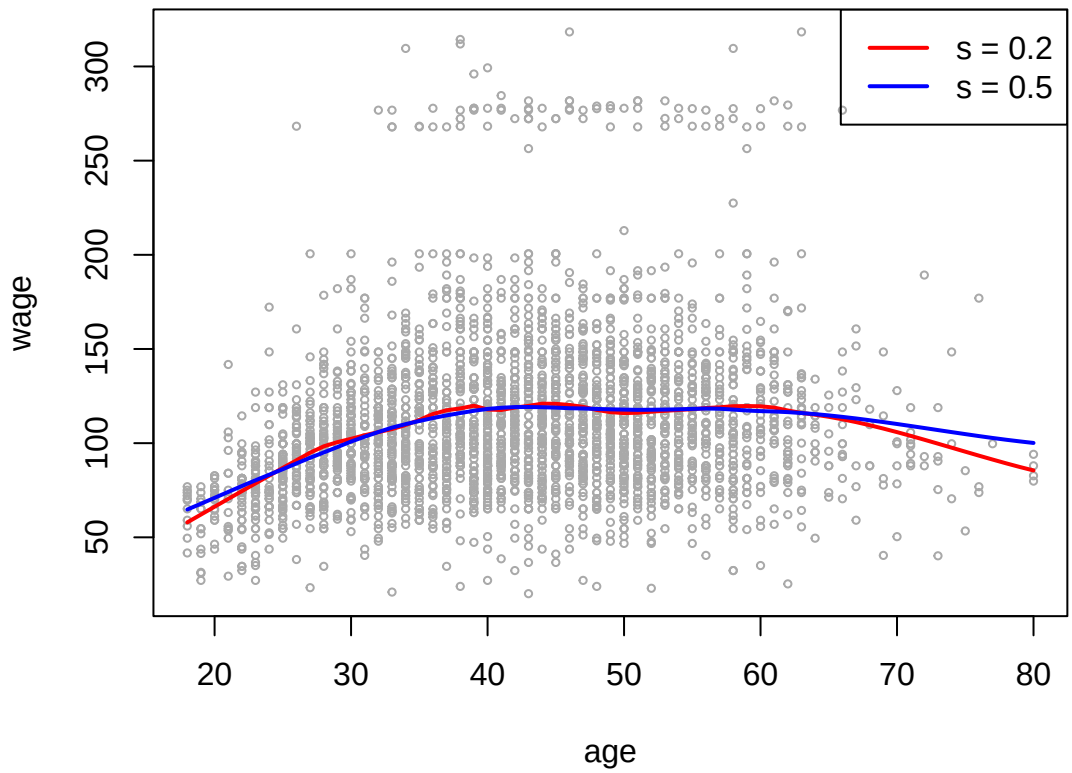
Domyślnie lokalne wielomiany stopnia 2; parametr `span` kontroluje szerokość okna.

```
spans <- c(0.2, 0.5)
clrs <- c("red", "blue")
plot(Wage$age, Wage$wage, cex = 0.5, col = "darkgrey", xlab = "age", ylab = "wage")
for (i in seq_along(spans)) {
  fit_loess <- loess(wage ~ age, span = spans[i], data = Wage)
  pred_loess <- predict(fit_loess, data.frame(age = age_grid))
  lines(age_grid, pred_loess, col = clrs[i], lwd = 2)
}
legend("topright", legend = paste("s =", spans), col = clrs, lty = 1, lwd = 2)
```



To samo z `degree = 1`:

```
plot(Wage$age, Wage$wage, cex = 0.5, col = "darkgrey", xlab = "age", ylab = "wage")
for (i in seq_along(spans)) {
  fit_loess <- loess(wage ~ age, span = spans[i], degree = 1, data = Wage)
  pred_loess <- predict(fit_loess, data.frame(age = age_grid))
  lines(age_grid, pred_loess, col = clrs[i], lwd = 2)
}
legend("topright", legend = paste("s =", spans), col = clrs, lty = 1, lwd = 2)
```



9 Uogólnione modele addytywne (GAM)

9.1 GAM dopasowany metodą najmniejszych kwadratów (lm + ns)

```
fit_gam_ls <- lm(
  wage ~ ns(year, df = 4) + ns(age, df = 5) + education,
  data = Wage
)
fit_gam_ls
```

```
##
## Call:
## lm(formula = wage ~ ns(year, df = 4) + ns(age, df = 5) + education,
##     data = Wage)
##
## Coefficients:
##                  (Intercept)            ns(year, df = 4)1
##                       46.949                        8.625
##            ns(year, df = 4)2            ns(year, df = 4)3
##                        3.762                        8.127
##            ns(year, df = 4)4             ns(age, df = 5)1
##                        6.806                       45.170
```

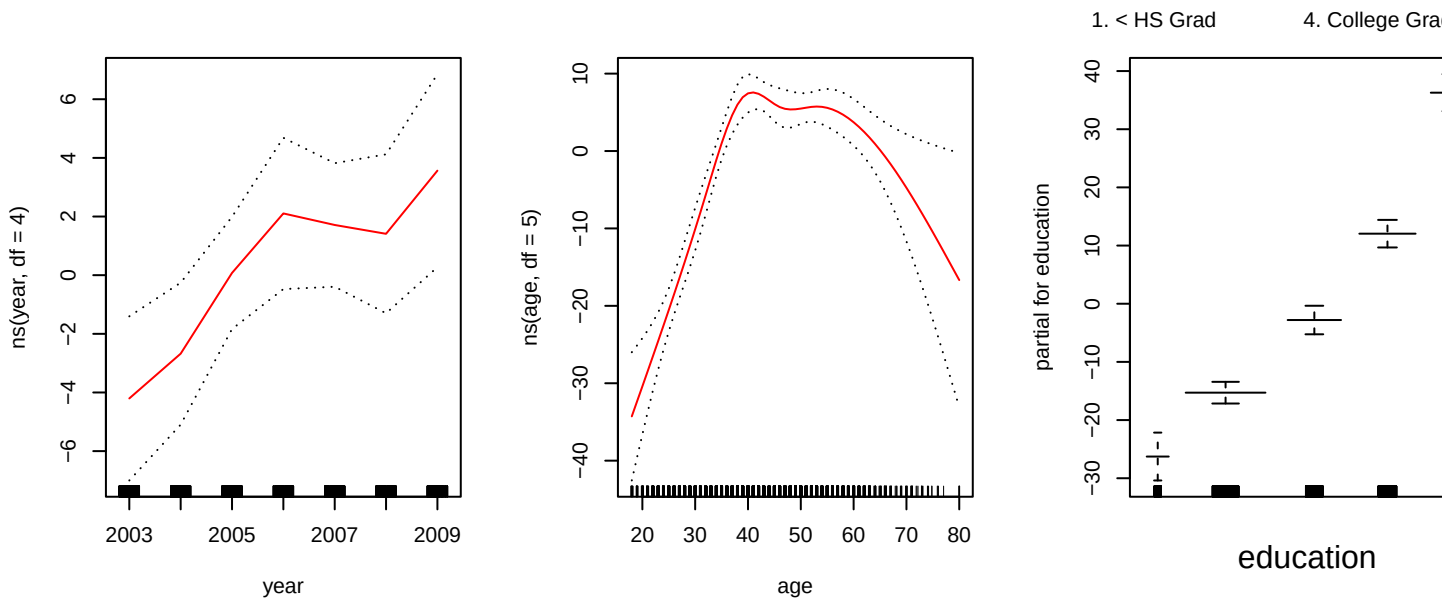
```
##            ns(age, df = 5)2             ns(age, df = 5)3
##                      38.450                       34.239
##            ns(age, df = 5)4             ns(age, df = 5)5
##                      48.678                        6.557
##         education2. HS Grad     education3. Some College
##                      10.983                       23.473
##    education4. College Grad  education5. Advanced Degree
##                      38.314                       62.554
```

```r
summary(fit_gam_ls)
```

```
##
## Call:
## lm(formula = wage ~ ns(year, df = 4) + ns(age, df = 5) + education,
##     data = Wage)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -120.513  -19.608   -3.583   14.112  214.535
##
## Coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                    46.949      4.704   9.980  < 2e-16 ***
## ns(year, df = 4)1               8.625      3.466   2.488  0.01289 *
## ns(year, df = 4)2               3.762      2.959   1.271  0.20369
## ns(year, df = 4)3               8.127      4.211   1.930  0.05375 .
## ns(year, df = 4)4               6.806      2.397   2.840  0.00455 **
## ns(age, df = 5)1               45.170      4.193  10.771  < 2e-16 ***
## ns(age, df = 5)2               38.450      5.076   7.575 4.78e-14 ***
## ns(age, df = 5)3               34.239      4.383   7.813 7.69e-15 ***
## ns(age, df = 5)4               48.678     10.572   4.605 4.31e-06 ***
## ns(age, df = 5)5                6.557      8.367   0.784  0.43328
## education2. HS Grad            10.983      2.430   4.520 6.43e-06 ***
## education3. Some College       23.473      2.562   9.163  < 2e-16 ***
## education4. College Grad       38.314      2.547  15.042  < 2e-16 ***
## education5. Advanced Degree    62.554      2.761  22.654  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 35.16 on 2986 degrees of freedom
## Multiple R-squared:  0.293,  Adjusted R-squared:  0.2899
## F-statistic:  95.2 on 13 and 2986 DF,  p-value: < 2.2e-16
```

Dla modelu z `lm` jawne wywołanie `plot.Gam` z pakietu `gam`:

```r
par(mfrow = c(1, 3))
plot.Gam(fit_gam_ls, col = "red", se = TRUE)
```




```r
par(mfrow = c(1, 1))
```

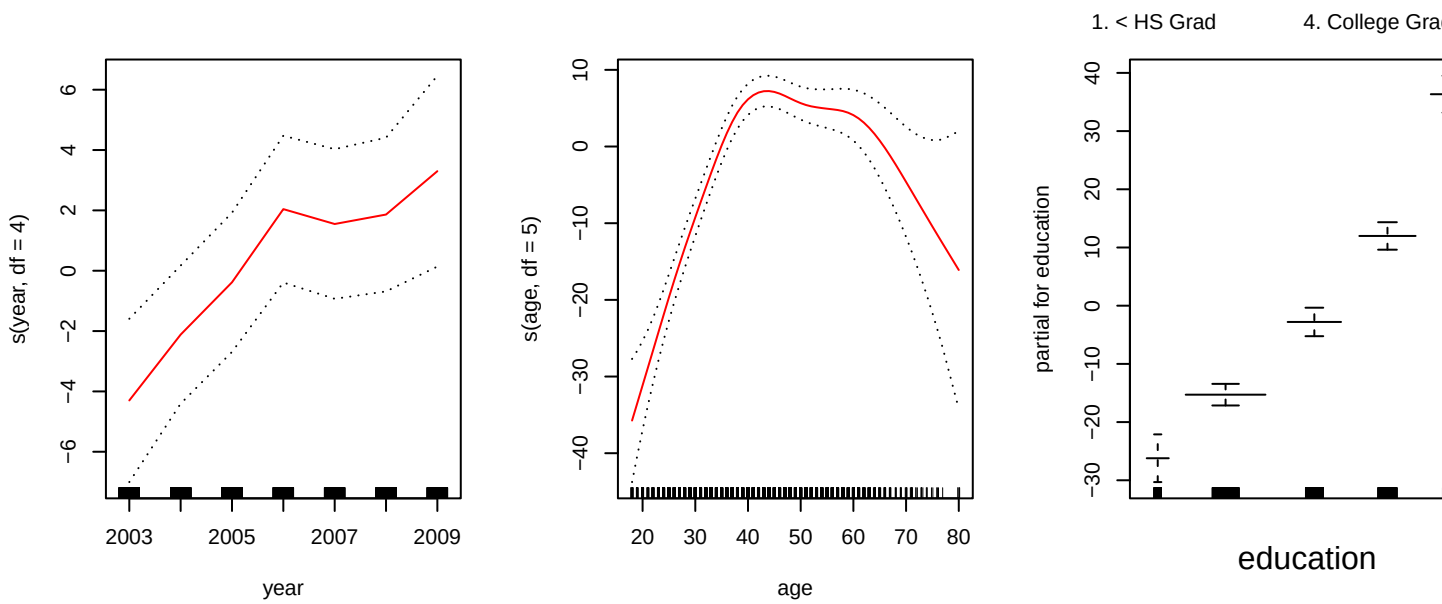
9.2 GAM — algorytm wsteczny, pakiet gam

```r
fit_gam_bf <- gam(wage ~ s(year, df = 4) + s(age, df = 5) + education, data = Wage)
summary(fit_gam_bf)
```

```
##
## Call: gam(formula = wage ~ s(year, df = 4) + s(age, df = 5) + education,
##     data = Wage)
## Deviance Residuals:
##     Min      1Q  Median      3Q     Max
## -119.43  -19.70   -3.33   14.17  213.48
##
## (Dispersion Parameter for gaussian family taken to be 1235.69)
##
##     Null Deviance: 5222086 on 2999 degrees of freedom
## Residual Deviance: 3689770 on 2986 degrees of freedom
## AIC: 29887.75
##
## Number of Local Scoring Iterations: NA
##
## Anova for Parametric Effects
##                   Df  Sum Sq Mean Sq F value    Pr(>F)
## s(year, df = 4)    1   27162   27162  21.981 2.877e-06 ***
## s(age, df = 5)     1  195338  195338 158.081 < 2.2e-16 ***
## education          4 1069726  267432 216.423 < 2.2e-16 ***
## Residuals       2986 3689770    1236
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Anova for Nonparametric Effects
```

```
##                   Npar Df Npar F  Pr(F)
## (Intercept)
## s(year, df = 4)       3  1.086 0.3537
## s(age, df = 5)        4 32.380 <2e-16 ***
## education
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
par(mfrow = c(1, 3))
plot(fit_gam_bf, col = "red", se = TRUE)
```




```r
par(mfrow = c(1, 1))
```

9.3 Porównanie modeli — anova

```r
fit_gam_1 <- gam(wage ~ s(age, df = 5) + education, data = Wage)
fit_gam_2 <- gam(wage ~ year + s(age, df = 5) + education, data = Wage)
anova(fit_gam_1, fit_gam_2, fit_gam_bf, test = "F")
```

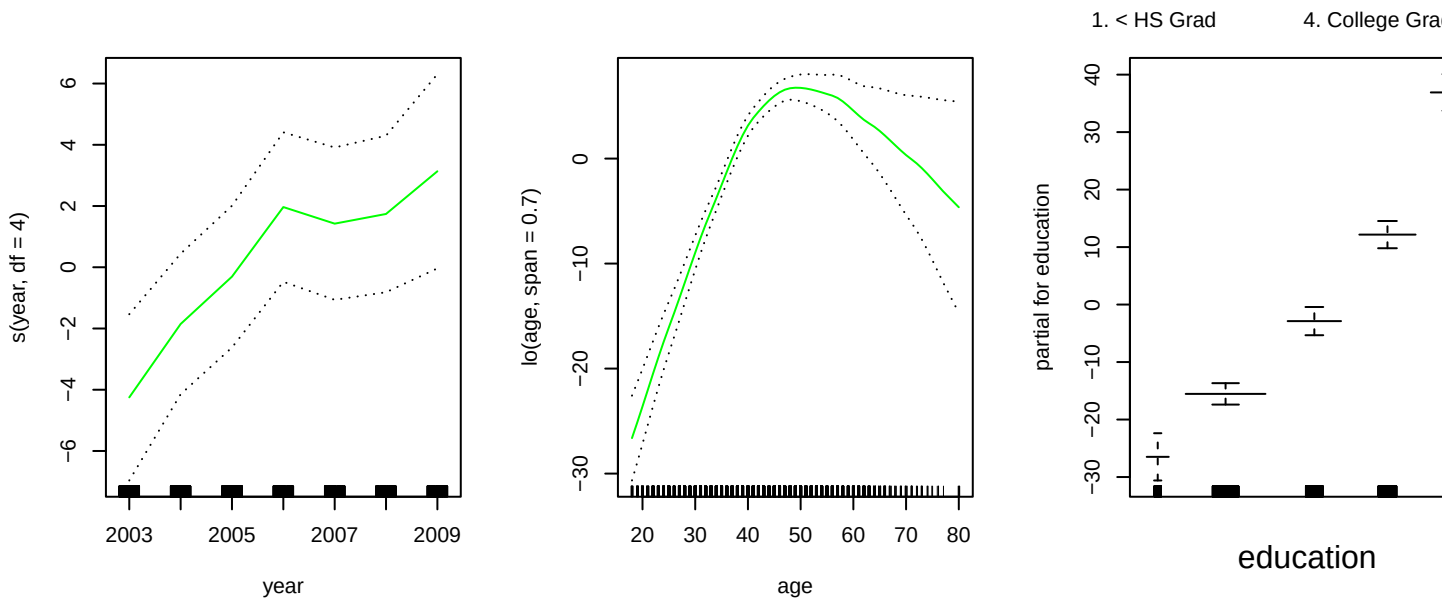
```
## Analysis of Deviance Table
##
## Model 1: wage ~ s(age, df = 5) + education
## Model 2: wage ~ year + s(age, df = 5) + education
## Model 3: wage ~ s(year, df = 4) + s(age, df = 5) + education
##   Resid. Df Resid. Dev Df Deviance       F    Pr(>F)
## 1      2990    3711731
## 2      2989    3693842  1  17889.2 14.4771 0.0001447 ***
## 3      2986    3689770  3   4071.1  1.0982 0.3485661
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

9.4 GAM z lokalną regresją `lo()`

```r
fit_gam_lo <- gam(
  wage ~ s(year, df = 4) + lo(age, span = 0.7) + education,
  data = Wage
)
summary(fit_gam_lo)
```

```
##
## Call: gam(formula = wage ~ s(year, df = 4) + lo(age, span = 0.7) +
##     education, data = Wage)
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -116.997  -19.319   -3.753   14.121  214.445
##
## (Dispersion Parameter for gaussian family taken to be 1243.534)
##
##     Null Deviance: 5222086 on 2999 degrees of freedom
## Residual Deviance: 3716672 on 2988.797 degrees of freedom
## AIC: 29903.95
##
## Number of Local Scoring Iterations: 1
##
## Anova for Parametric Effects
##                          Df  Sum Sq Mean Sq F value    Pr(>F)
## s(year, df = 4)        1.0   25188   25188  20.255 7.037e-06 ***
## lo(age, span = 0.7)    1.0  195537  195537 157.243 < 2.2e-16 ***
## education              4.0 1101825  275456 221.511 < 2.2e-16 ***
## Residuals           2988.8 3716672    1244
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Anova for Nonparametric Effects
##                     Npar Df Npar F  Pr(F)
## (Intercept)
## s(year, df = 4)         3.0  1.103 0.3464
## lo(age, span = 0.7)     1.2 88.835 <2e-16 ***
## education
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
par(mfrow = c(1, 3))
plot(fit_gam_lo, col = "green", se = TRUE)
```




```r
par(mfrow = c(1, 1))
```

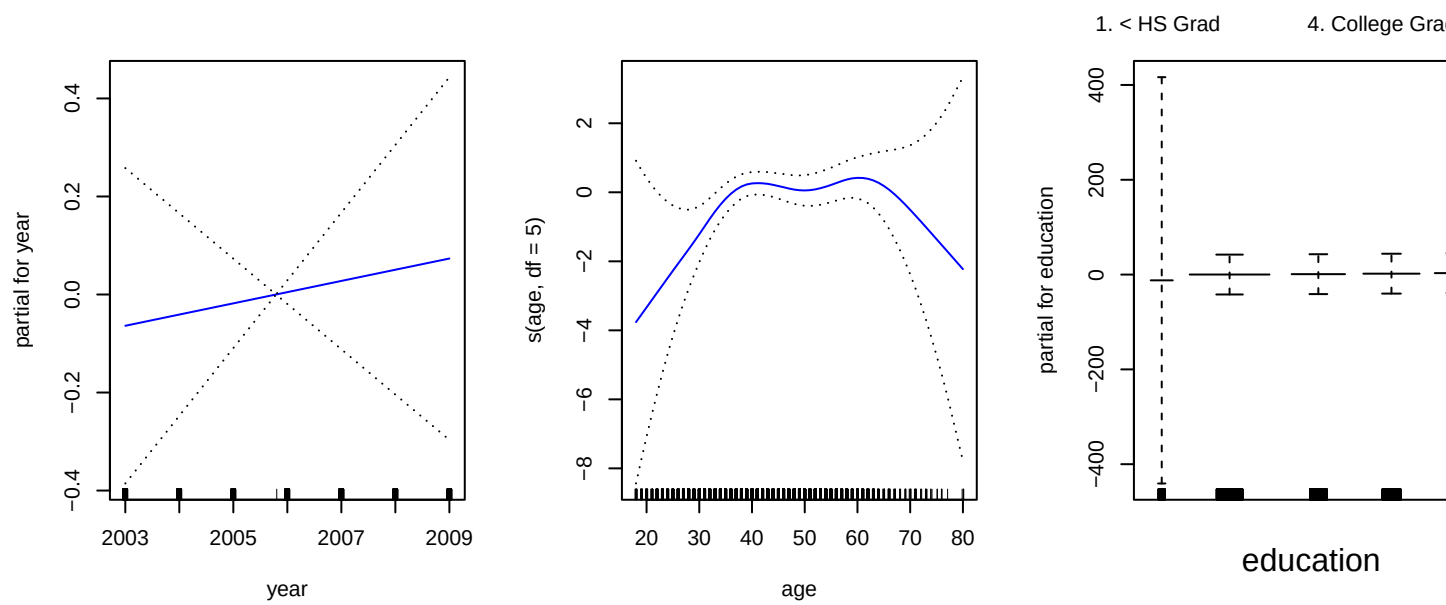
10 GAM w GLM — regresja logistyczna

```r
fit_logistic_gam <- gam(
  I(wage > 250) ~ year + s(age, df = 5) + education,
  family = binomial,
  data = Wage
)
summary(fit_logistic_gam)
```

```
##
## Call: gam(formula = I(wage > 250) ~ year + s(age, df = 5) + education,
##     family = binomial, data = Wage)
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -0.58206 -0.26780 -0.12341 -0.08241  3.31242
##
## (Dispersion Parameter for binomial family taken to be 1)
##
##     Null Deviance: 730.5345 on 2999 degrees of freedom
## Residual Deviance: 602.4588 on 2989 degrees of freedom
## AIC: 624.4586
##
## Number of Local Scoring Iterations: NA
##
## Anova for Parametric Effects
##                  Df  Sum Sq Mean Sq F value  Pr(>F)
## year              1    0.48  0.4845  0.5995 0.43883
## s(age, df = 5)    1    3.83  3.8262  4.7345 0.02964 *
## education         4   65.81 16.4514 20.3569 < 2e-16 ***
```

```
## Residuals      2989 2415.55  0.8081
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Anova for Nonparametric Effects
##                Npar Df Npar Chisq  P(Chi)
## (Intercept)
## year
## s(age, df = 5)       4     10.364 0.03472 *
## education
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
par(mfrow = c(1, 3))
plot(fit_logistic_gam, col = "blue", se = TRUE)
```




```r
par(mfrow = c(1, 1))
```